

POWER BI DATA MODELLING

What to Learn & How — A Complete Learning Guide

Covering: Star Schema | DAX | Relationships | Power Query | Performance | Best Practices

Prepared for: Pathikrit Roy

Edition: 2025-2026

TABLE OF CONTENTS

Power BI Data Modelling — Complete Learning Guide

Intro Introduction: Why Data Modelling is the Foundation of Power BI

Prerequisites | How to Use This Guide

Ch 1 The Big Picture — What Is Data Modelling?

- 1.1 The Three Layers of Power BI
 - 1.2 What a Data Model Actually Contains
 - 1.3 The Two Modes: Import vs DirectQuery
-

Ch 2 Power Query — Shaping Data for the Model

- 2.1 The Power Query Editor
 - 2.2 Essential Transformations to Master
 - 2.3 The M Language — Key Concepts
 - 2.4 Power Query Best Practices
-

Ch 3 Dimensional Modelling — The Star Schema

- 3.1 What Is a Star Schema?
 - 3.2 Star Schema Example: Retail Sales
 - 3.3 Why Not Just Use One Big Table?
 - 3.4 The Date Dimension — Non-Negotiable
-

Ch 4 Relationships — Connecting Your Tables

- 4.1 Relationship Properties
 - 4.2 Creating Relationships
 - 4.3 Role-Playing Dimensions
 - 4.4 Many-to-Many Relationships
-

Ch 5 DAX — The Formula Language

- 5.1 Measures vs Calculated Columns
 - 5.2 Filter Context and Row Context
 - 5.3 Essential DAX Functions
 - 5.4 Variables in DAX
-

Ch 6 Time Intelligence and Advanced Patterns

- 6.1 Prerequisites for Time Intelligence
 - 6.2 Core Time Intelligence Functions
 - 6.3 Fiscal Year Time Intelligence
 - 6.4 Ranking and Top N Patterns
 - 6.5 Cumulative Total Pattern
-

Ch 7 Performance Optimisation

- 7.1 How VertiPaq Works
- 7.2 Performance Golden Rules
- 7.3 The Performance Analyser
- 7.4 DAX Studio — Your Performance Debug Tool

Ch 8 Best Practices and Common Mistakes

- 8.1 Naming Conventions
- 8.2 Measure Organisation
- 8.3 The 10 Most Common Beginner Mistakes
- 8.4 Row Level Security (RLS)

Ch 9 Your Learning Roadmap

- 9.1 Skill Levels
- 9.2 Week-by-Week Study Plan
- 9.3 Microsoft PL-300 Certification
- 9.4 Resources

Introduction: Why Data Modelling is the Foundation of Power BI

Power BI is one of the most widely used business intelligence tools in the world. But here is the truth most beginners miss: the charts and visuals are only 20% of the work. The other 80% is data modelling — the invisible infrastructure that determines whether your reports are fast, accurate, and maintainable, or slow, wrong, and impossible to update.

A well-designed data model transforms a messy collection of Excel sheets and database tables into a clean, performant analytical system. It is the difference between a report that refreshes in 2 seconds and one that takes 2 minutes, between a DAX formula that is 5 lines long and one that is 50.

What You Will Learn in This Guide

Chapter 1: The Big Picture — what data modelling is and why it matters
Chapter 2: Power Query — transforming raw data into model-ready tables
Chapter 3: Star Schema and Dimensional Modelling — the right shape for analytics
Chapter 4: Relationships — connecting your tables correctly
Chapter 5: DAX — the formula language that powers calculations
Chapter 6: Advanced Modelling Patterns — time intelligence, many-to-many, roles
Chapter 7: Performance Optimisation — making models fast at scale
Chapter 8: Best Practices and Common Mistakes — what separates pros from beginners
Chapter 9: Your Learning Roadmap — step-by-step skill progression

Prerequisites

This guide assumes you can open Power BI Desktop and have some familiarity with Excel. No programming background is required. All concepts are explained from first principles.

How to Use This Guide

Read each chapter in order. Every chapter has a Hands-On Exercise at the end. Do not skip them — data modelling is a practical skill that you learn by doing, not by reading alone. Download the free sample datasets from Microsoft's GitHub (search 'Power BI sample datasets') to follow along.

Chapter 1: The Big Picture — What Is Data Modelling?

Data modelling is the process of designing how your tables connect to each other, how your data is structured, and how your business calculations are defined. Think of it as designing the blueprint of a building before construction begins.

1.1 The Three Layers of Power BI

Layer	Tool	Purpose
Data Layer	Power Query (M)	Connect, clean, and shape raw data from sources
Model Layer	Model View + DAX	Define relationships, hierarchies, measures
Report Layer	Report View	Visualise model data with charts and tables

Most beginners spend 90% of their time in the report layer. Professionals spend 60% of their time in the model and data layers. The quality of your report is entirely determined by the quality of your model underneath.

1.2 What a Data Model Actually Contains

- Tables: the raw data organised into rows and columns
- Relationships: connections between tables (joins) defined in the model
- Measures: DAX calculations (revenue, profit margin, YoY growth) stored in the model
- Calculated Columns: row-level DAX computations added as new columns to a table
- Hierarchies: drill-down paths (Year > Quarter > Month > Day)
- Roles: row-level security rules controlling who sees which data

1.3 The Two Modes: Import vs DirectQuery

Mode	Characteristics
Import (Default)	Data is copied into Power BI's in-memory engine (VertiPaq). Fastest performance. Data refreshes on schedule (hourly at most on Power BI Service). Best for datasets under 1GB.
DirectQuery	Every visual sends a query live to the source database. Always current data. Slower. Source DB must handle the load. Use for real-time requirements or very large data.
Composite Model	Mix Import and DirectQuery tables in the same model. Advanced use case.
Streaming	Real-time dashboards for IoT/event data. Tiles update in seconds. Limited analytical capability.

RECOMMENDATION: Start with Import mode for all learning and most production scenarios. Only move to DirectQuery if you have a specific need (real-time, data >1GB, governance).

Chapter 2: Power Query — Shaping Data for the Model

Power Query (also called the M language) is where you connect to data sources and transform raw, messy data into clean tables ready for modelling. Good data modelling starts with good Power Query work.

2.1 The Power Query Editor

Open it from Home > Transform Data. The editor has four key areas: the Queries pane (left), the Data Preview (centre), the Query Settings panel (right) with Applied Steps, and the Formula Bar.

Every transformation you apply creates a new step in Applied Steps. Power Query is non-destructive — your source data is never modified. Steps are reapplied on every refresh.

2.2 Essential Transformations to Master

Transformation	When to Use	How to Apply
Remove Columns	Drop columns you will never use (reduces model size)	Right-click column header > Remove
Rename Columns	Use clear, consistent names (no spaces, no special chars)	Double-click header
Change Data Type	Ensure dates are Date, numbers are Decimal/Integer, not Text	Click type icon in header
Filter Rows	Remove nulls, test data, future dates not yet useful	Column header dropdown
Remove Duplicates	Create clean dimension tables	Home > Remove Rows > Remove Duplicates
Group By	Aggregate fact data at the right grain	Transform > Group By
Pivot / Unpivot	Convert wide format to long (unpivot for month columns)	Transform > Unpivot Columns
Merge Queries	Join two tables (like SQL JOIN) to enrich data	Home > Merge Queries
Append Queries	Stack tables vertically (like SQL UNION ALL)	Home > Append Queries
Custom Column	Add a column using M language formula	Add Column > Custom Column
Split Column	Separate 'First Last' into First and Last	Transform > Split Column
Replace Values	Fix inconsistent entries ('N/A', 'na', '#N/A' -> null)	Transform > Replace Values

2.3 The M Language — Key Concepts

Power Query uses the M language under the hood. You do not need to write M from scratch, but reading and editing it makes you far more powerful.

Anatomy of an M Query

```
let
    Source = Csv.Document(File.Contents("C:\data\sales.csv"), [Delimiter=","]),
    Promoted = Table.PromoteHeaders(Source),
    TypedTable = Table.TransformColumnTypes(Promoted, {
        {"Date", type date}, {"Revenue", type number}, {"Units", Int64.Type}
    }),
    FilteredRows = Table.SelectRows(TypedTable, each [Revenue] > 0),
    RemovedCols = Table.RemoveColumns(FilteredRows, {"Internal_ID", "Temp_Flag"})
in
    RemovedCols
```

Useful M Functions

Function	Purpose
Table.SelectRows(t, each [col] = val)	Filter rows by condition
Table.AddColumn(t, 'Name', each [A] + [B])	Add a custom computed column
Text.Trim([col])	Remove leading/trailing whitespace
Text.Upper([col]) / Text.Lower([col])	Normalise string casing
Date.Year([DateCol])	Extract year from a date
List.Contains({list}, [col])	Check membership in a list (like SQL IN)
Table.Join(t1, 'key', t2, 'key', JoinKind.Left)	Merge with explicit join type
Number.RoundDown([col], 2)	Round to 2 decimal places

2.4 Power Query Best Practices

- **Load only what you need:** remove unused columns in Power Query, not DAX. Every column in the model consumes memory.
- **Type everything:** always set data types explicitly. Power BI's auto-detection is unreliable for dates with unusual formats.
- **Name queries clearly:** use PascalCase — FactSales, DimCustomer, DimDate. Not 'Sheet1' or 'Table1'.
- **Separate staging queries:** use reference queries to share transformation steps. Right-click a query > Reference.
- **Never load staging tables to model:** right-click query > Enable Load = OFF for intermediate steps.
- **Avoid complex transformations in DAX:** if it can be done in Power Query, do it there. Power Query transforms happen once on refresh; DAX runs on every visual.

Hands-On Exercise: Load the Adventure Works sales CSV. Remove all columns except Date, Product, Region, Revenue, Units. Change types. Remove rows where Revenue is null or zero. Name the query FactSales.

Chapter 3: Dimensional Modelling — The Star Schema

The star schema is the single most important concept in data modelling for analytics. It is not a Power BI-specific concept — it was developed for data warehouses in the 1990s and remains the gold standard for analytical systems today.

3.1 What Is a Star Schema?

A star schema has two types of tables: Fact tables (what happened) and Dimension tables (who, what, when, where). The fact table sits in the centre; dimension tables surround it like points of a star.

Fact Tables

A fact table records business events at the lowest grain needed. Each row is one event — one sale, one transaction, one page view. Fact tables contain: foreign keys to dimension tables and numeric measures (Revenue, Quantity, Cost).

Grain is everything. Decide the grain first: 'Each row = one order line item' or 'Each row = one daily sales summary per store.' Mixing grains in one table is a common and catastrophic mistake.

Dimension Tables

Dimension tables contain descriptive attributes about the entities in your facts. They are typically narrow (few columns) but wide (many descriptive attributes). Examples: DimCustomer (CustomerID, Name, City, Country, Segment), DimProduct (ProductID, Name, Category, SubCategory, Brand).

3.2 Star Schema Example: Retail Sales

Imagine a retailer with this business question: 'What was total revenue by product category, by region, for Q3 this year, for customers with gold membership?'

Table	Columns
FactSales (Fact)	SalesKey, DateKey, ProductKey, CustomerKey, StoreKey, Revenue, Units, Cost
DimDate (Dimension)	DateKey, Date, Year, Quarter, Month, MonthName, WeekNum, IsWeekend
DimProduct (Dimension)	ProductKey, ProductName, Category, SubCategory, Brand, Cost
DimCustomer (Dimension)	CustomerKey, CustomerName, City, Country, Segment, MembershipTier
DimStore (Dimension)	StoreKey, StoreName, City, Region, Country, OpenDate

Each of the four business question dimensions maps directly to a dimension table. The question is answered by filtering dimensions and summing Revenue in the fact table.

3.3 Why Not Just Use One Big Table?

New Power BI users often flatten everything into a single wide table. This is called a flat table or denormalised approach. It seems simpler but creates serious problems:

Problem	Impact
Data redundancy	Customer name stored once per transaction. 1M sales = 1M copies of customer names. Huge file size.
Update anomalies	Change a product category — must update 100K rows instead of 1 row in DimProduct.
Slow DAX	Relationship-based filtering is far faster than filtering large flat tables in the engine.
Many-to-many headaches	Flat tables cannot express complex relationships without row duplication.
No reusability	Cannot share dimension tables across multiple fact tables (e.g., DimDate used by both FactSales and FactBudget).

RULE: If a value in your fact table appears on hundreds of rows unchanged (CustomerCity, ProductCategory), it belongs in a dimension table. Move it out.

3.4 The Date Dimension — Non-Negotiable

Every Power BI model that analyses trends over time **MUST** have a dedicated Date dimension table (DimDate). Do not rely on Power BI's auto date/time hierarchy — it creates hidden tables for every date column and kills performance.

Creating a Date Table in DAX

```
DimDate =
VAR StartDate = DATE(2020, 1, 1)
VAR EndDate = DATE(2026, 12, 31)
RETURN
ADDCOLUMNS(
    CALENDAR(StartDate, EndDate),
    "Year",          YEAR([Date]),
    "Quarter",       "Q" & QUARTER([Date]),
    "Month",         MONTH([Date]),
    "MonthName",     FORMAT([Date], "MMMM"),
    "MonthShort",    FORMAT([Date], "MMM"),
    "WeekNum",       WEEKNUM([Date]),
    "DayOfWeek",     WEEKDAY([Date], 2),
    "DayName",       FORMAT([Date], "DDDD"),
    "IsWeekend",     IF(WEEKDAY([Date], 2) >= 6, TRUE, FALSE),
    "FiscalYear",    IF(MONTH([Date]) >= 4, YEAR([Date]), YEAR([Date]) - 1),
    "YearMonth",     FORMAT([Date], "YYYY-MM"),
    "YYYYMMDD",     YEAR([Date]) * 10000 + MONTH([Date]) * 100 + DAY([Date])
)
```

After creating DimDate: mark it as a Date Table (Table Tools > Mark as Date Table, select the Date column). This is required for time intelligence DAX functions (SAMEPERIODLASTYEAR, DATESYTD, etc.) to work correctly.

Chapter 4: Relationships — Connecting Your Tables

Relationships tell Power BI how tables are connected. They are the mechanism through which filter context flows — when a user selects 'Q3' on a slicer in the report, that selection propagates through relationships to filter the fact table.

4.1 Relationship Properties

Property	Options and Meaning
Cardinality	One-to-Many (*:1): standard for dim-to-fact. One-to-One (1:1): same grain, use sparingly. Many-to-Many (*:*): avoid in most cases — use a bridge table.
Cross Filter Direction	Single: filter flows one way (dimension -> fact). Both: bidirectional. Use Both sparingly — it causes ambiguity and slows the engine.
Active vs Inactive	Only one active relationship per table pair. Inactive relationships used via USERRELATIONSHIP() in DAX. Common for role-playing dimensions.

4.2 Creating Relationships

- Drag a column from one table to another in Model View
- Or use Manage Relationships > New
- The join column should have the same data type on both sides
- Ideally, join on integer surrogate keys, not text strings (faster engine matching)

GOLDEN RULE: Relationships should always flow from dimension (one side) to fact (many side). Filter propagates from '1' to '*'. Never create a relationship in the wrong direction.

4.3 Role-Playing Dimensions

A role-playing dimension is a single dimension table used multiple times with different roles. The most common example: a Date table used for Order Date, Ship Date, and Due Date — all linking to the same fact table.

Power BI allows only one active relationship between two tables. So: make the primary relationship active (Order Date) and create additional inactive relationships for Ship Date and Due Date.

```
// DAX Measure: Revenue by Ship Date
Shipped Revenue =
CALCULATE(
    [Total Revenue],
    USERRELATIONSHIP(FactSales[ShipDateKey], DimDate[DateKey])
)
```

4.4 Many-to-Many Relationships

Avoid native many-to-many (*:*) relationships wherever possible. They are slow and produce counterintuitive results. Instead, use a bridge table.

Bridge Table Pattern

Example: A customer can belong to multiple segments, and a segment can have multiple customers. Create a bridge table CustomerSegment with CustomerKey and SegmentKey. Then connect: DimCustomer (1) -> BridgeCustomerSegment (*) -> DimSegment (1).

```
// Set both relationships to Single cross-filter direction
// Then in DAX, use TREATAS to handle the many-to-many correctly
Revenue by Segment =
CALCULATE(
    [Total Revenue],
    TREATAS(
        VALUES(DimSegment[SegmentKey]),
        BridgeCustomerSegment[SegmentKey]
    )
)
```

Chapter 5: DAX — The Formula Language

DAX (Data Analysis Expressions) is the formula language for Power BI, Analysis Services, and Power Pivot. It is used to create two things: Measures (aggregations evaluated in filter context) and Calculated Columns (row-level computations stored in the table).

5.1 Measures vs Calculated Columns

	Measures	Calculated Columns
Evaluated when	On demand, when a visual renders	At data refresh, stored physically
Context	Filtered by slicers, visuals, row context	No filter context — uses row context
Storage	No storage — dynamic calculation	Stored in memory, adds to model size
Use for	Aggregations: SUM, AVERAGE, ratios	Attributes: ProfitMargin = Revenue - Cost
Best practice	Prefer measures for all aggregations	Only for attributes needed as filters/slicers

The fundamental rule: if the calculation needs filter context (slicers, visual filters), it must be a measure. Calculated columns cannot respond to filter context.

5.2 Filter Context and Row Context

Understanding context is the single most important concept in DAX. Every DAX expression is evaluated within a context that determines which rows are visible.

Filter Context

Created by slicers, visual filters, page filters, and report filters. When you drop a measure into a visual, the filter context is the combination of all filters applied to that visual. CALCULATE() is the only function that can modify filter context.

Row Context

Created when iterating over a table — inside SUMX, AVERAGEX, ADDCOLUMNS, calculated columns. Row context knows the current row but does NOT automatically filter. Use RELATED() to bring in values from related dimension tables during row iteration.

5.3 Essential DAX Functions

Basic Aggregations

```
Total Revenue = SUM(FactSales[Revenue])
Average Order Value = AVERAGE(FactSales[Revenue])
Unique Customers = DISTINCTCOUNT(FactSales[CustomerKey])
Max Sale = MAX(FactSales[Revenue])
Transaction Count = COUNTROWS(FactSales)
```

CALCULATE — The Most Important DAX Function

CALCULATE evaluates an expression in a modified filter context. It accepts any number of filter modifiers. Every advanced DAX formula uses CALCULATE.

```
// Revenue for a specific region
UK Revenue = CALCULATE([Total Revenue], DimStore[Country] = "UK")

// Revenue for Gold customers only
Gold Revenue = CALCULATE([Total Revenue], DimCustomer[Tier] = "Gold")

// REMOVEFILTERS: ignore all filters on a column
Total All Regions = CALCULATE([Total Revenue], REMOVEFILTERS(DimStore[Region]))

// % of Total
Revenue % of Total = DIVIDE([Total Revenue], [Total All Regions], 0)
```

SUMX and Iterator Functions

Iterator functions (SUMX, AVERAGEX, MAXX, MINX) iterate row by row over a table, evaluate an expression per row, and then aggregate the results.

```
// Gross Profit: must compute per row THEN sum (cannot SUM(Revenue - Cost))
Gross Profit = SUMX(FactSales, FactSales[Revenue] - FactSales[Cost])

// Weighted Average Selling Price
Avg Selling Price = DIVIDE(
    SUMX(FactSales, FactSales[Revenue]),
    SUMX(FactSales, FactSales[Units]),
    0
)
```

FILTER Function

FILTER returns a filtered table. Use inside CALCULATE for complex conditions that cannot be expressed as a simple column = value filter.

```
// Revenue from transactions over £1000
Large Deal Revenue = CALCULATE(
    [Total Revenue],
    FILTER(FactSales, FactSales[Revenue] > 1000)
)
```

PERFORMANCE NOTE: Prefer CALCULATE(measure, column = value) over CALCULATE(measure, FILTER(table, column = value)) when possible. Direct column filters are much faster than FILTER() on large tables.

RELATED and RELATEDTABLE

```
// In FactSales, get the Category from DimProduct (many-to-one)
Product Category = RELATED(DimProduct[Category]) // Calculated column

// In DimCustomer, count transactions (one-to-many)
Customer Transactions = COUNTROWS(RELATEDTABLE(FactSales)) // Calculated column
```

IF, SWITCH, and Logical Functions

```
// Tier Label
Revenue Tier =
SWITCH(
    TRUE(),
```

```
[Total Revenue] >= 1000000, "Platinum",  
[Total Revenue] >= 500000, "Gold",  
[Total Revenue] >= 100000, "Silver",  
"Bronze"  
)  
  
// Safe division (avoid divide by zero)  
Profit Margin % = DIVIDE([Gross Profit], [Total Revenue], 0)
```

5.4 Variables in DAX

Variables (VAR/RETURN) make DAX readable and performant. They evaluate once and can be reused. Always use variables in complex measures.

```
Revenue Growth % =  
VAR RevenueCY = [Total Revenue]  
VAR RevenuePY = [Revenue Previous Year]  
VAR Growth    = DIVIDE(RevenueCY - RevenuePY, RevenuePY, BLANK())  
RETURN  
    IF(RevenuePY = 0, BLANK(), Growth)
```

BEST PRACTICE: Use VAR for any value you reference more than once in a measure. It makes the formula faster (calculated once) and much easier to debug.

Chapter 6: Time Intelligence and Advanced Patterns

Time intelligence is the ability to compare periods — year-over-year growth, year-to-date totals, rolling averages. It is one of the most common analytical requirements and one of the most powerful capabilities in DAX.

6.1 Prerequisites for Time Intelligence

- A marked Date table (Table Tools > Mark as Date Table)
- Continuous date range — no gaps in dates
- An active relationship between FactSales[DateKey] and DimDate[DateKey]

6.2 Core Time Intelligence Functions

```
// Year-to-Date
Revenue YTD = CALCULATE([Total Revenue], DATESYTD(DimDate[Date]))

// Quarter-to-Date
Revenue QTD = CALCULATE([Total Revenue], DATESQTD(DimDate[Date]))

// Month-to-Date
Revenue MTD = CALCULATE([Total Revenue], DATESMTD(DimDate[Date]))

// Same period last year
Revenue PY = CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(DimDate[Date]))

// Year-over-Year growth
Revenue YoY % = DIVIDE([Total Revenue] - [Revenue PY], [Revenue PY], BLANK())

// Previous month
Revenue PM = CALCULATE([Total Revenue], PREVIOUSMONTH(DimDate[Date]))

// Last 3 months rolling
Revenue L3M =
CALCULATE(
    [Total Revenue],
    DATESINPERIOD(DimDate[Date], LASTDATE(DimDate[Date]), -3, MONTH)
)
```

6.3 Fiscal Year Time Intelligence

If your financial year starts on a month other than January (e.g., April), use DATESYTD with the end-of-year parameter.

```
// Fiscal Year starts April 1 (FY ends March 31)
Revenue FYTD = CALCULATE([Total Revenue], DATESYTD(DimDate[Date], "03-31"))

// Previous Fiscal Year
Revenue Prior FY = CALCULATE([Total Revenue],
    DATEADD(DATESYTD(DimDate[Date], "03-31"), -1, YEAR))
```


6.4 Ranking and Top N Patterns

```
// Rank products by Revenue
Product Rank =
RANKX(
    ALL(DimProduct[ProductName]),
    [Total Revenue],
    ,
    DESC,
    DENSE
)

// Show only Top 10 products
// Add this as a visual-level filter: [Product Rank] <= 10
```

6.5 Cumulative Total Pattern

```
Cumulative Revenue =
CALCULATE(
    [Total Revenue],
    FILTER(
        ALL(DimDate[Date]),
        DimDate[Date] <= MAX(DimDate[Date])
    )
)
```

Chapter 7: Performance Optimisation

A poorly optimised Power BI model can be painfully slow — reports taking 30+ seconds to render, or timing out entirely. Understanding how the VertiPaq engine works is the key to building fast models.

7.1 How VertiPaq Works (Simplified)

Power BI's in-memory engine, VertiPaq, stores data column by column (columnar storage) and compresses each column using dictionary encoding and run-length encoding. The fewer unique values in a column, the better the compression. This is why removing high-cardinality columns (GUIDs, free-text notes) dramatically reduces model size.

7.2 Performance Golden Rules

Rule	Why It Matters
Remove unused columns	Every column consumes memory regardless of whether it's used
Remove unused rows	Filter to the date range you actually report on (last 3 years, not all history)
Avoid high-cardinality text columns	Unique strings like email addresses, free text notes = terrible compression
Use integer surrogate keys for relationships	Integer joins are dramatically faster than string joins
Avoid calculated columns for aggregations	Use measures instead — they don't expand the model size
Minimise bidirectional relationships	Each bidirectional relationship increases query complexity exponentially
Avoid complex iterators on large tables	SUMX on 50M rows without filtering first = slow
Use DIVIDE not /, 0	Built-in null handling, slightly faster than IF(denominator=0,BLANK(),a/b)
Disable Auto Date/Time	File > Options > Data Load > Disable Auto Date/Time globally

7.3 The Performance Analyser

Use View > Performance Analyser to measure how long each visual takes to render. It breaks down: DAX Query time (time computing the measure), Visual Display time (rendering), and Other.

- If DAX Query is high: optimise your measures with DAX Studio
- If Visual Display is high: reduce the number of data points in the visual
- If Other is high: consider reducing visual complexity or number of visuals per page

7.4 DAX Studio — Your Performance Debug Tool

DAX Studio (free, from daxstudio.org) connects to your Power BI model and allows you to: run DAX queries directly, view the query plan, measure execution time, and use Server Timings to see Storage Engine vs Formula Engine time.

Key insight: VertiPaq Storage Engine queries are fast and parallelisable. Formula Engine queries are slower and single-threaded. Optimising DAX means pushing as much work as possible to the Storage Engine.

Chapter 8: Best Practices and Common Mistakes

8.1 Naming Conventions

Consistent naming makes models maintainable across teams and over time.

Object	Convention	Example
Fact tables	FactXxx	FactSales, FactBudget, FactInventory
Dimension tables	DimXxx	DimProduct, DimCustomer, DimDate
Bridge tables	BridgeXxx	BridgeCustomerSegment
Measures	No prefix, Title Case	Total Revenue, Gross Margin %
Hidden measures	Prefix with _	_Base Revenue (used inside other measures)
Calculated columns	No prefix	ProfitMargin, ProductFullName
Display folders	Group related measures	'Revenue Measures', 'Time Intelligence'

8.2 Measure Organisation

- Group all measures in a dedicated, data-free table: Home > Enter Data, name it _Measures, load as empty table
- Use Display Folders in Model View to group measures: Revenue, Profitability, Time Intelligence
- Write a description for every measure (right-click > Properties) — this shows as tooltip in reports
- Sort date columns correctly (Format > Sort by Column — sort MonthName by Month number)

8.3 The 10 Most Common Beginner Mistakes

Mistake	Correct Approach
Flat table instead of star schema	Separate into fact + dimension tables
No Date table / using Auto Date Time	Create a DimDate, mark it as date table, disable Auto Date/Time
Bidirectional relationships everywhere	Use single direction; use CROSSFILTER() in DAX when needed
Measures that reference other measures inside CALCULATE incorrectly	Understand context transition; use CALCULATE to wrap correctly
Using calculated columns where measures would work	Measures respond to filter context; columns do not
Not using DIVIDE for division	DIVIDE handles division-by-zero; / will cause errors

Mistake	Correct Approach
Loading all historical data	Filter to relevant date range in Power Query
Using FILTER(ALL(table),...) when a simpler filter works	Use column filters directly in CALCULATE arguments
Inconsistent date formats causing relationship failures	Standardise all dates to Date type in Power Query
Mixing grains in one fact table	Separate grains into separate fact tables

8.4 Row Level Security (RLS)

RLS restricts which rows of data a user sees in published reports. Define roles in Power BI Desktop, then assign users to roles in Power BI Service.

```
// RLS Role: Sales Reps see only their region
// In Modelling > Manage Roles, add a filter to DimStore:
[SalesRepEmail] = USERPRINCIPALNAME()

// Or for hierarchical security using PATHCONTAINS:
PATHCONTAINS(DimOrg[Hierarchy], LOOKUPVALUE(DimOrg[OrgID], DimOrg[Email],
USERPRINCIPALNAME()))
```

Test RLS in Power BI Desktop: Modelling > View as Role. Always test all roles before publishing.

Chapter 9: Your Learning Roadmap

Power BI data modelling is a skill with clear levels. Here is a structured progression from absolute beginner to advanced practitioner, with honest time estimates for each level.

9.1 Skill Levels

Level	Skills and Duration
Level 1: Foundations (Weeks 1-2)	Connect to Excel/CSV, basic Power Query transforms, simple star schema, active relationships, basic DAX: SUM, COUNT, AVERAGE, simple CALCULATE
Level 2: Intermediate (Weeks 3-5)	Full Power Query workflow, proper Date table, time intelligence (YTD, YoY), SUMX iterators, DIVIDE, VAR/RETURN, basic RLS
Level 3: Advanced (Weeks 6-10)	Complex DAX patterns (semi-additive, many-to-many bridge tables), TREATAS, RANKX, composite models, performance optimisation, DAX Studio
Level 4: Expert (Months 3-6)	Large-scale model design, incremental refresh, deployment pipelines, C2L sensitivity labels, Power BI Embedded, Tabular Editor, ALM Toolkit

9.2 Week-by-Week Study Plan

Week	Focus and Activities
Week 1	Install Power BI Desktop. Complete Microsoft's 'Get started with Power BI' learning path (free on learn.microsoft.com). Connect to an Excel file, explore Power Query.
Week 2	Study Chapter 2 of this guide. Practise: load 3 different data sources, apply 10 different transformations, create a staging query workflow.
Week 3	Study Chapter 3. Build your first star schema from a flat sales file. Create DimDate using the DAX formula in this guide.
Week 4	Study Chapters 4-5. Create all relationship types. Write 20 DAX measures covering SUM, CALCULATE, SUMX, DIVIDE, IF/SWITCH, VAR.
Week 5	Study Chapter 6. Implement all time intelligence measures: YTD, QTD, YoY %, rolling 3-month. Publish to Power BI Service.
Week 6	Study Chapters 7-8. Use Performance Analyser on your report. Install DAX Studio. Reduce model size by removing unused columns.
Week 7	Build a complete end-to-end project: Adventure Works or Contoso dataset (free downloads). Star schema + 30+ measures + RLS.
Week 8	Study advanced patterns: many-to-many with bridge tables, role-playing dimensions, RANKX Top N. Complete the PL-300 practice tests.

9.3 Microsoft PL-300 Certification

The PL-300: Microsoft Power BI Data Analyst is the industry-recognised certification for Power BI. It validates your ability to prepare data, model data, visualise data, and deploy/maintain assets. Achieving PL-300 signals professional-level competence to employers.

Exam Domain	Weight
Prepare the Data	25-30%
Model the Data	25-30%
Visualise and Analyse the Data	25-30%
Deploy and Maintain Assets	15-20%

Data Modelling accounts for the largest single exam domain. Mastering this guide thoroughly gives you the knowledge for roughly 50-55% of the exam content (model + prepare).

9.4 Resources

Free

- Microsoft Learn: learn.microsoft.com/power-bi — official learning paths, all free
- SQLBI.com — the world's best DAX learning resource. Guy in a Cube on YouTube for practical Power BI
- 'The Definitive Guide to DAX' by Marco Russo & Alberto Ferrari — the DAX bible
- Power BI Community forum — search before you ask; most questions already answered

Paid

- Pragmatic Works training — enterprise Power BI courses
- Udemy: 'Microsoft Power BI Desktop for Business Intelligence' by Maven Analytics
- SQLBI DAX courses — advanced, highly recommended once you have the basics

Practice Datasets

- Adventure Works (Microsoft) — classic retail dataset, perfect for learning
- Contoso (Microsoft) — larger retail dataset, good for performance practice
- Northwind Traders — smaller, simpler, good for beginners
- Your own company data — always the best motivator

Final Advice: The fastest way to learn Power BI data modelling is to take a real business question you care about, find the data, and build the model to answer it. Theory without practice stalls. Practice without theory produces bad models. Do both.